
인공지능을 위한 선형대수학

실습 3. 최소제곱법으로 예측 모델 만들기



오늘 할 일

이론을 코드로 검증합니다

- 정확한 해가 **없는** 문제를 직접 확인 — `solve` 가 실패합니다
- 정규방정식으로 최소제곱해를 구합니다
- **정말 최소인지** 답을 흔들어 확인합니다
- **정말 직교하는지** 내적으로 확인합니다

SECTION 1

해가 없다는 것 확인하기

No Exact Solution

4명 데이터 입력

Over-determined System

[1] 따라 쓰기

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
A = np.array([[60, 5.5, 1],
              [65, 5.0, 0],
              [55, 6.0, 1],
              [50, 5.0, 1]]) # 4번째 사람 추가
b = np.array([66, 74, 78, 72])
print(A.shape, b.shape)
```

실행 결과

(4, 3) (4,)



solve 는 실패합니다

Deliberate Error

[2]

따라 쓰기

```
# 일부러 에러를 내는 셀입니다  
print(np.linalg.solve(A, b))
```


SECTION 2

어떤 답이 더 좋은가

Comparing Guesses

두 답의 오차 비교

Which is Better?

[3] [4]

따라 쓰기

```
x_old = np.array([-0.4, 20, -20])      # 2장의 답
print(b - A @ x_old, np.linalg.norm(b - A @ x_old))
x_guess = np.array([-0.12, 16, -9.5])  # 찍어본 답
print(b - A @ x_guess, np.linalg.norm(b - A @ x_guess))
```

실행 결과

```
[ 0.  0.  0. 12.] 12.0
[-5.3  1.8 -1.9  7.5] 9.5493
```


SECTION 3

정규방정식으로 풀기

Solving the Normal Equation

4×3 을 3×3 으로

$A^T A$ and $A^T b$

[5]

따라 쓰기

```
C = A.T @ A
d = A.T @ b
print(C)
print("크기:", C.shape)
print("d =", d)
```

실행 결과

```
[[13350.    1235.    165.  ]
 [ 1235.    116.25    16.5  ]
 [  165.     16.5     3.   ]]
크기: (3, 3)
d = [16660.  1561.  216.]
```

최소제곱해

The Least Squares Solution

[6] [7]

따라 쓰기

```
x_hat = np.linalg.solve(C, d)
for name, x in [("2장의 답", x_old), ("찍어본 답", x_guess), ("최소제곱해", x_hat)]:
    print(name, np.linalg.norm(b - A @ x))
```

실행 결과

$\hat{x} = [-0.1523 \quad 16.5836 \quad -10.8327]$

2장의 답 12.0

찍어본 답 9.5493

최소제곱해 9.5059

SECTION 4

정말 최소인가?

Verification

답을 흔들어 봅시다

Perturbation Test

[8]

따라 쓰기

```
base = np.linalg.norm(b - A @ x_hat)
for i in range(3):
    for step in (-0.5, 0.5):
        x_try = x_hat.copy(); x_try[i] += step
        print(i, step, np.linalg.norm(b - A @ x_try))
```

실행 결과

기준	9.505945	
x[0] ±0.5	58.547955	더 큼
x[1] ±0.5	10.928197	더 큼
x[2] ±0.5	9.545312	더 큼

직교 조건 확인

$$A^T(b - A\hat{x}) = 0$$

[9]

따라 쓰기

```
r = b - A @ x_hat          # 오차(잔차) 벡터
print("r      =", r)
print("A^T r =", A.T @ r)
print(np.allclose(A.T @ r, 0))
```

실행 결과

```
r      = [-5.2384  0.9822 -2.2918  7.5302]
A^T r = [-0. -0. -0.]
True
```

실무에서는 한 줄

np.linalg.lstsq

[10]

따라 쓰기

```
x_lstsq = np.linalg.lstsq(A, b, rcond=None)[0]
print(x_lstsq)
print(np.allclose(x_lstsq, x_hat))
```

실행 결과

```
[ -0.1523  16.5836 -10.8327]
```

```
True
```

데이터를 40명으로

Scaling Up

[11] [12] 따라 쓰기

```
rng = np.random.default_rng(0)
n = 40
A40 = np.column_stack([rng.uniform(45, 90, n),
                        rng.uniform(4.8, 6.2, n),
                        rng.integers(0, 2, n)])

true_x = np.array([-0.15, 16.5, -10.5]) # 진짜 규칙
b40 = A40 @ true_x + rng.normal(0, 1.0, n) # 잡음 섞기
print(np.linalg.lstsq(A40, b40, rcond=None)[0])
```

실행 결과

진짜 규칙 : [-0.15 16.5 -10.5]

찾아낸 답 : [-0.16 16.605 -10.367]

직접 해보기

- 1 수업 확인 문제를 코드로 풀고 손 계산과 대조하시오.
 $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \rightarrow A.T@A, A.T@b, \hat{x}, \text{오차 길이 출력 (답: } \hat{x} = [1/3, 1/3])$
- 2 문제 1의 오차 벡터가 A 의 두 열과 직교함을 코드로 확인하시오.
- 3 40명 데이터에서 잡음을 0으로 바꾸면 (`rng.normal(0, 0.0, n)`) 찾아낸 답이 `true_x`와 얼마나 가까워지는지 확인하고, 그 이유를 텍스트 셀에 쓰시오.

오늘 배운 명령어

Summary

코드	뜻
<code>A.T @ A , A.T @ b</code>	정규방정식 세우기
<code>solve(A.T@A, A.T@b)</code>	정규방정식 풀기
<code>lstsq(A, b, rcond=None)[0]</code>	최소제곱해 한 줄로 — 실무 표준
<code>norm(b - A@x)</code>	오차의 길이
<code>np.column_stack([...])</code>	열을 옆으로 붙여 행렬 만들기
<code>np.random.default_rng(0)</code>	결과가 항상 같은 난수 생성기